# Can AI Run Its Own Experiments? An Artifact-Based Replication and Reliability Evaluation of AutoRecLab

Javier Briceño Ticona
University of Siegen
Siegen, Germany

Dylan Fandio
University of Siegen
Siegen, Germany

Jawdat Alqawi
University of Siegen
Siegen, Germany

## ABSTRACT

AutoRecLab is a proof-of-concept LLM multi-agent system that turns a natural-language description of a recommender-systems experiment into a structured specification, executable code, and results, coordinated by a planner, a coder, and a reviewer under a tree search. In its original random-seed case study, Beel et al. reported that three of four runs completed the requested experiment (75%) and that 56% of the generated subexperiments were valid—already indicating that completion and correctness diverge. We evaluate AutoRecLab's reliability at the level of its retained artifacts. Our evidence spans 36 historical replication runs, for which mostly only aggregate outcomes survive. Additionally, 33 follow-up searches were conducted: four were executed within the development environment and partially retained, while 29 were fully retained. The latter were executed using AutoRecLab `v1.0.0`—19 exploratory and 10 preregistered—whose every file we inventoried, hashed, and inspected, including all intermediate tree-search nodes. Applying a six-level criterion from code generation to complete semantic validity, no follow-up search produced a coherent, semantically valid 270-entry result matrix. A human-authored positive control nonetheless completed the same matrix in 49 minutes at no model cost, so the failures are not attributable to task infeasibility. In a preregistered, matched comparison on one model, the intervention did not improve completion, and the observed mean best reviewer score was lower under P3 than under P2, and the validator was reinterpreted by generated code rather than enforced—one report rewrote its own denominator from 270 to 30 and self-passed. AutoRecLab's reviewer score also tracked objective coverage only weakly: one search scored 81.8% while covering two of 270 requested entries. We conclude that execution, output presence, and automated reviewer approval are insufficient evidence of a valid experiment, and that reliable autonomous experimentation requires external, non-editable semantic validation.

## KEYWORDS

AutoRecLab, LLM agents, recommender systems, reproducibility, autonomous experimentation, replication, semantic validity

## 1 INTRODUCTION

### 1.1 Background

Recommender systems are a cornerstone of modern information filtering, powering personalized content delivery in domains ranging from e-commerce and streaming services to academic literature and social media [1]. The research process surrounding these systems, however, remains largely manual and labor-intensive: researchers formulate hypotheses, design experiments, implement pipelines, tune hyperparameters, and interpret results, repeating similar workflows across studies. In parallel, machine learning and artificial intelligence have seen growing interest in automating scientific workflows, with large language models (LLMs) increasingly used for code generation, experimental design, and even the synthesis of findings. Despite these advances, the recommender-systems community has been comparatively slow to automate its own research pipeline end to end: established tools such as Auto-Surprise, LensKit-Auto, and LibRec-Auto address only narrow sub-tasks such as algorithm selection and hyperparameter optimization. This gap motivates more comprehensive automation frameworks that support the full lifecycle of a recommender-systems experiment, from problem specification to result interpretation.

### 1.2 Research Problem

Beel et al. introduced AutoRecLab as a proof-of-concept system for automating recommender-systems experiments, demonstrating that LLM-based agents can translate natural-language prompts into executable pipelines [1]. That evidence, however, rests on four runs and a single random-seed case study, and 44% of its subexperiments produced invalid results—most prominently a systematic misconfiguration of ItemKNN for implicit feedback that was identified but not resolved. Two properties of this evidence make independent scrutiny necessary. First, run completion does not guarantee methodological correctness: a search can terminate normally, and its reviewer can approve the outcome, while the underlying computation deviates from the requested protocol. Second, the failing ItemKNN subexperiments raised no runtime error, so the defect was silent—an output that looked successful but was not. Establishing whether AutoRecLab is reliable therefore requires evaluating the retained artifacts themselves, the generated code, execution logs, and native result files, rather than trusting completion counts or the system's self-assessment. This need is compounded by a practical hazard: the artifacts of large replication efforts are easily lost, leaving only aggregate outcomes that can no longer be re-examined at the level of individual experiments.

### 1.3 Original Work

Beel et al. [1] argue that the recommender-systems community has paid little attention to automating the research *process* itself. Although tools such as Auto-Surprise, LensKit-Auto, and LibRec-Auto exist, they are limited to algorithm selection and hyperparameter tuning and do not generate research questions, design experiments, or produce findings. The authors therefore call for an AutoRecLab: an integrated system, or set of interoperable tools, able to support the full research workflow for recommender systems.

To explore this idea, they built a proof-of-concept that takes a natural-language description of an experiment and realizes three sequential roles through prompt engineering on a single LLM. A

*planner* translates the prompt into a structured experiment specification and a checklist of binary methodological requirements; a *coder* generates and executes the corresponding Python pipeline; and a *reviewer* scores the implementation against the checklist. A tree-search algorithm drives an iterative loop that generates, executes, and repairs code, selecting the highest-scoring implementation at each step while occasionally exploring alternative branches to escape local optima.

To evaluate the system, the authors reproduced a prior human study of how random data-splitting seeds affect recommendation accuracy. The task used three algorithms (ALS, ItemKNN, and Most Popular), three datasets (MovieLens 100K, Last.FM, and Amazon Video Games), and five random seeds under an 80/20 user-based holdout split, with performance measured by nDCG@$k$ and Precision@$k$ for $k \in \{1, 5, 10\}$. The same experiment was run four times with slightly varied prompts, at a budget of 10–20 tree-search iterations per subexperiment, an API cost of $1–$5 per run, and runtimes of 2–9.5 hours on a consumer laptop.

Three of the four runs completed successfully. Across them, AutoRecLab produced 27 subexperiments, of which 15 were valid (56%). The 12 invalid outputs were concentrated in two diagnosable failure modes: all nine ItemKNN subexperiments failed because of the implicit-feedback misconfiguration, and three Most Popular subexperiments in a single run were also invalid. Because the ItemKNN error raised no exception, the tree search did not detect it. For the valid cases, mainly ALS and Most Popular, the outputs closely matched the human baseline and supported the same conclusions. The authors conclude that AutoRecLab can already produce reviewer-relevant experimental evidence at low additional cost, that its failure modes are concentrated and diagnosable rather than random, and they propose a roadmap of open prototypes, benchmarks, and reproducibility standards for AI-assisted experimentation.

### 1.4 Research Goal

Our goal is to evaluate AutoRecLab's reliability independently and at the level of evidence integrity, combining the reported outcomes of a large historical replication phase with a detailed artifact-level analysis of a separately retained set of follow-up runs executed in the final AutoRecLab `v1.0.0` environment. Rather than asking only whether a search completed, we ask whether its retained artifacts demonstrate that the requested experiment—its datasets, preprocessing, splits, algorithms, seeds, metrics, and the full 270-entry result matrix—was actually carried out. We assess progress with a graded criterion ranging from code generation through exception-free execution and native-metric production to complete semantic validity, and we compare AutoRecLab's own reviewer score against this objective evidence.

## 2 METHODOLOGY

### 2.1 Study Design and Experimental Phases

We evaluated AutoRecLab through repeated search runs under different prompt, language-model, and software configurations. A *run* denotes one invocation with a fixed prompt and configuration, from initialization until normal termination, timeout, or failure. Dataset–algorithm–seed combinations generated within an invocation are outputs of that run and are not counted as independent runs.

**Table 1: Historical , Explorative, and Comparison Phases**

| Phase | Runs | Full | Partial | Usage |
|---|---|---|---|---|
| Historical Replication | 36 | — | 8 | Reported completion rates |
| Explorative Follow-up | 23 | 19 | 4 (dev) | Artifact analysis |
| Preregistered P2–P3 Comparison | 10 | 10 | 0 | Intervention comparison |

The initial evaluation comprised 36 runs on the original `main`-branch configuration, which specified LensKit 0.14.4. The surviving records do not independently establish an exact commit and dependency state for every run: 33 used a detailed prompt and three used a simplified prompt. Aggregate completion outcomes were retained for the 36 initial runs. Eight runs additionally have partially recovered supporting records; consequently, the 36-run comparison is limited to reported completion outcomes, while artifact-level conclusions rely on the separately retained audit sample. Seven of these records consist primarily of bounded log evidence, while one recovered record contains an intermediate checkpoint and a 45-row result bundle covering all three datasets, three named algorithms, and five seeds under the simplified prompt. Because that prompt did not specify metric cutoffs, the bundle has one row per dataset-algorithm-seed combination with both metrics stored as columns. It is therefore substantial evidence of a complete simplified-prompt output, but it is not the 270-entry detailed-protocol matrix and its semantic validity cannot be fully reconstructed. The historical phase is therefore used for aggregate completion comparisons; artifact-level claims are restricted to the evidence actually retained.

To extend this evidence, we conducted 23 additional planned production runs. Nineteen fully retained runs were executed in the final environment and form the principal artifact-audited sample. Four additional runs were executed on a second machine using a `develop`-branch implementation. Because their complete run workspaces were not available in the audited repository corpus, they are reported separately and are used only for claims supported by their preserved records.

The following phase was a preregistered comparison designed to test whether an explicit compatibility and output-validation contract improved the reliability of autonomous experiment execution. It comprised 10 additional production runs on GPT-5.4 Mini: five repetitions of the detailed compatibility prompt P2 (R01–R05) and five repetitions of the contract-validated prompt P3 (R01–R05). The two conditions were fixed in advance and used identical model, software state, iteration budget, timeout, datasets, preprocessing requirements, algorithms, cutoffs, and seeds; P3 differed from P2 only by the appended compatibility and deterministic validation requirements. Each repetition was launched as an independent invocation with a fresh output directory. This third phase therefore contributes 10 production runs, bringing the total study corpus to 69 reported production runs across three phases: 36 historical runs, 23 exploratory follow-up runs, and 10 preregistered comparison runs. Because the phases differ in software state, artifact availability, and evidentiary depth, these totals are reported as a corpus inventory rather than as a single homogeneous evaluation sample.

**Table 2: Allocation of the 33 additional production runs across the exploratory and preregistered phases. "Audited" denotes runs with fully retained workspaces; `develop` denotes the four separately documented `develop`-branch runs.**

| Prompt | Model | Audited | `develop` | Total |
|---|---|---|---|---|
| *Exploratory* | | | | |
| P0 | GPT-5 Nano | 5 | 0 | 5 |
| P1 | GPT-5 Nano | 5 | 0 | 5 |
| P0 | GPT-5.4 Mini | 5 | 2 | 7 |
| P0 | GPT-5.4 | 2 | 2 | 4 |
| P2 | GPT-5.4 | 2 | 0 | 2 |
| *Preregistered* | | | | |
| P2 | GPT-5.4 Mini | 5 | 0 | 5 |
| P3 | GPT-5.4 Mini | 5 | 0 | 5 |
| Total | | 29 | 4 | 33 |

## 2.2 Experimental Setup and Differences from the Original Study

We targeted the recommender-system experiment described in the original AutoRecLab study. The datasets were MovieLens 100K [3], Amazon Video Games [5], and Last.FM from HetRec 2011 [2]. MovieLens and Amazon ratings strictly greater than three were converted to implicit interactions. Last.FM tag events were treated as implicit user–artist interactions and repeated pairs were collapsed. Each dataset was then subjected to iterative five-core filtering.

The requested algorithms were alternating least squares (ALS), item-based nearest neighbours (ItemKNN), and Most Popular. For every dataset and algorithm, five user-based random 80/20 train–test splits were requested. Performance was measured using nDCG@$k$ and Precision@$k$ for $k \in \{1, 5, 10\}$. A complete run therefore contained 270 unique, finite metric entries:

$$3 \text{ datasets} \times 3 \text{ algorithms} \times 5 \text{ seeds} \times 3 \text{ cutoffs} \times 2 \text{ metrics}.$$

P0 specified this protocol in detail and requested LensKit 0.14.4. P1 retained the overall task but omitted requirements including the rating threshold, five-core procedure, and fixed cutoffs. P2 retained the detailed protocol but used the packages installed with AutoRecLab `v1.0.0` at upstream commit `60e7d2a`, fixed the seeds to 7, 13, 29, 42, and 87, required the supplied local files and their actual schemas, prohibited substitute downloads and a separate validation split, and rejected mock, placeholder, or non-finite results. Exact phase-specific prompt texts are included in the reproducibility materials described in Section 2.7.

Table 2 reports the allocation of the 23 additional production runs. The four runs executed on the second machine are shown separately from the 19 fully audited runs so that run count is not confused with artifact availability.

## 2.3 Feasibility Positive Control

To separate failures of the autonomous system from possible infeasibility of the requested protocol, one author implemented the full experiment by hand under the final AutoRecLab `v1.0.0` environment (OmniRec 1.0.0, LensKit 2025.6.2). The control uses the same datasets, preprocessing, seeds, split, algorithms, metrics, and cutoffs as the specification, and its outputs are checked by an independent deterministic validator that verifies all 270 keys, finite values, the nDCG@1–Precision@1 identity, and the preprocessing and split counts. It is a methodological control, not an additional benchmark and not a comparison against AutoRecLab's recommendation quality.

## 2.4 Preregistered Compatibility Contract Intervention

After the exploratory runs, we ran a preregistered, balanced comparison on a single model (GPT-5.4 Mini): five repetitions of the detailed prompt P2 and five of an intervention prompt P3. P3 is byte-identical to P2 except for two appended sections: an environment compatibility contract that states the known incompatibilities of the installed stack, and a deterministic output-and-validation contract that requires a 270-entry result table and a machine-readable validation report. The prompt text, code-base commit, model, iteration budget, timeout, datasets, and seeds (7, 13, 29, 42, and 87) were fixed before running and left unchanged across repetitions, and success was predeclared as a validator-confirmed 270/270 matrix with zero semantic violations. Some repetitions ran concurrently on one machine, so runtime and peak-memory comparisons for this phase are descriptive; cost, coverage, semantic validity, and completion are unaffected.

## 2.5 Software, Hardware, and Repositories

We used the implementation released by the original authors [4] rather than reimplementing its search procedure. The team fork is identified in the final reproducibility release [6]. The historical phase is associated with commit `2df2bdf`, which uses LensKit 0.14.4 without OmniRec. The four additional runs executed on the second machine used OmniRec 1.0.0 and LensKit 2025.6.2; repository history suggests a state near commit `edb02a`, but the retained records do not establish the exact commit. These runs are therefore not treated as software-equivalent to the 19 fully retained runs executed at `f71623f`, based on the official AutoRecLab `v1.0.0` commit `60e7d2a`, with OmniRec 1.0.0 and LensKit 2025.6.2. Results from these software states are reported separately.

The configuration used for the 19 fully retained runs generated two draft nodes, allowed five search iterations and two refinement iterations, and set the generated-program timeout to 5,400 seconds. The language-model temperature was 1.0, with a 120-second request timeout and at most three request retries. Resolved configuration files were stored with each run. Raw datasets were supplied under `study/data`; the release supplies their source URLs and SHA-256 checksums (`study/study_ context`) rather than redistributing the datasets.

Experiments ran on two Windows gaming laptops and one Windows gaming desktop. The first machine, which was used to execute the 19 fully retained runs, ran Windows 11 Pro and had an Intel Core i7-10750H CPU, 32,GiB RAM, and an NVIDIA GeForce GTX 1650 GPU. The second machine was a desktop with an AMD Ryzen 5 7500F six-core CPU, 32,GiB RAM, and an AMD Radeon RX 6750 XT GPU with 12,GiB VRAM. The third machine was a laptop with a 12th-generation Intel Core i7-12700H CPU, 16,GiB RAM,

and an NVIDIA GeForce RTX 4060 Laptop GPU with 8,GiB VRAM, alongside integrated Intel Iris Xe graphics. Energy consumption was not instrumented.

### 2.6 Artifact Audit, Success Criteria, and Error Classification

The audit of the primary corpus covered every retained file under `study_runs` and `study_logs`: 2,810 files totaling 47.47 GiB. Each file was inventoried and hashed bytewise with SHA-256. Generated Python code, text and console logs, CSV files, and JSON files were parsed or inspected; large prediction JSON files were processed with streaming logic. Pickles and other binary formats were checked for readability and expected signatures but were not deserialized. Hash-based duplicate detection prevented copied checkpoints and deterministic reruns from being treated as independent evidence.

For each of the 29 fully retained production searches including the ten preregistered P2–P3 runs, we examined its prompt, resolved configuration, requirement checklists, generated code, execution logs, native OmniRec `results.json` files, derived outputs, cost log, and run summary. Automated markers located exceptions, crashes, downloads, placeholders, and mock or fallback logic. Candidate cases were then checked by comparing the relevant code, log, and result artifacts. All intermediate nodes were inspected to avoid relying only on AutoRecLab's final selection.

Progress was evaluated using increasingly strict criteria: code generation, execution without a recorded exception, output production, requested native metric production, semantic validity, and complete validity. For the OmniRec-based runs in the primary audit corpus, native coverage required a finite requested metric in `results.json`. A semantically valid entry additionally required the specified dataset, preprocessing, user-based split, algorithm, seed, cutoff, and metric, with a value computed from actual recommendations rather than a mock, fallback, placeholder, or relabelled result. Complete validity required all 270 unique keys within one coherent node or continued execution; rows from independently generated or incompatible nodes were never combined to create an apparently complete run. The eight partially recovered records from the initial evaluation were assessed using the canonical outputs available in their earlier software state, not the OmniRec-specific file criterion.

AutoRecLab's requirement score and `is_satisfactory` marker were recorded as properties of its self-evaluation, not as ground truth. Observed problems were coded into non-exclusive execution errors (split or validation, schema or loader, download or path, framework API, evaluation schema, and memory), semantic errors (preprocessing or split deviations, substituted data or algorithms, mislabeled metrics, mocks, fallbacks, non-finite values, and incomplete matrices), and search-orchestration errors such as selecting a weaker final node. Ambiguous cases were resolved by triangulating code, logs, and structured results; the categories are therefore used as a qualitative taxonomy rather than mutually exclusive event counts.

### 2.7 Reproducibility and AI Assistance

The final repository release [1] accompanying this paper includes a directory organized into: machine-readable audit tables for the exploratory and preregistered phases; the audit and figure scripts; the phase-specific prompts, dataset source URLs, and SHA-256 checksums; curated case-study artifacts with representative code, logs, native results, and validation reports; and the human-authored positive-control pipeline with its external validator. A per-file SHA-256 manifest is included for the complete 47.47 GiB raw workspaces, which are not stored in Git. Most artifacts from the historical 36-run phase are unavailable, and the aggregate historical results cannot be verified at the same depth as the retained follow-up runs.

Generative-AI tools assisted with code and audit-script development, artifact exploration, analysis organization, and language editing. AI output was not treated as evidence. The authors checked claims against the underlying code, logs, structured results, and cited sources and remained responsible for the experimental design, verification, interpretation, and final text.

[1] https://github.com/dylanfandio/AutoRecLab/releases/tag/study-v1.0.0

## 3 RESULTS & DISCUSSION

### 3.1 Original Study and Replication Outcomes

The original AutoRecLab study reported that three of four search runs completed the requested experiment (75%) and that 56% of the generated subexperiments were valid [1]—already indicating that run completion and subexperiment correctness were not equivalent. These figures are the reference point for our replication, but they rest on four runs and the original study's own assessment.

The historical replication phase did not reproduce the reported run-level completion rate. None of the 33 P0 detailed-prompt runs was reported as successful, whereas one of the three P1 simplified-prompt runs was reported as successful. This corresponds to 0% and 33.3% (2.8% across all 36 runs). The simplified-prompt success is an aggregate historical record whose artifact chain is unavailable, so it cannot be retested against the criteria in Section 2.6; the recovered records provide supporting execution evidence. In particular, one intermediate node produced a complete 45-row output for the simplified prompt, but the available artifacts are insufficient to verify it against the detailed protocol or to identify it conclusively as the historically reported satisfactory run.

In the retained AutoRecLab `v1.0.0` follow-up phase, none of the 19 production searches yielded a coherent, semantically valid 270-entry result matrix, and none received a true `is_satisfactory` marker. Four additional runs executed on a second machine used the `develop`-branch environment and remain a separate evidence tier outside the 19-run denominator. The phases do not share one success definition, but the original 75% result transferred to neither the historical replication nor the artifact-verified follow-up (Figure 1).

### 3.2 Artifact-Verified Completion and Partial Progress

The absence of a complete valid run does not mean that the searches produced no working experimental code. The 19 retained production searches generated 188 code nodes. Of these, 78 completed without a recorded exception (41.5%), while 110 recorded an exception or crash (58.5%). The nodes are nested attempts within 19 tree searches rather than 188 independent experiments. Moreover, 24 of the 78 non-crashing nodes contained mock or fallback behavior. A node could therefore terminate without an exception while

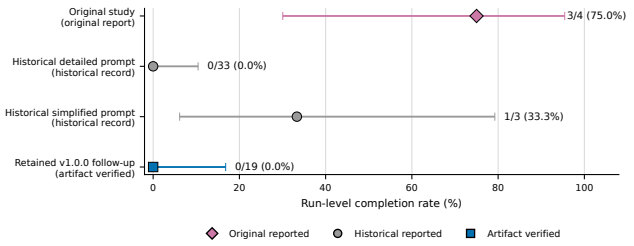


**Figure 1: Run-level completion across the original study and the two replication phases; horizontal lines show 95% Wilson intervals. Success definitions and evidence quality differ per tier, so the rates are a contextual rather than a controlled comparison. The four additional `develop`-branch runs lack an equivalent denominator and are omitted.**

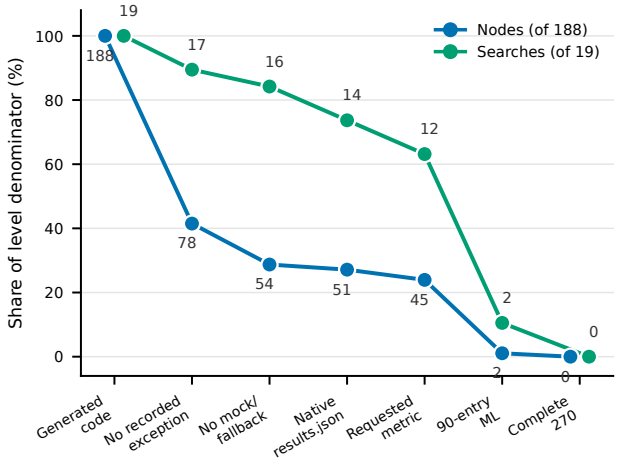


**Figure 2: Share of the 188 generated nodes and 19 production searches reaching each evidence criterion; labels give absolute counts. A search counts when at least one of its nodes provides the evidence. 'Native result file' means that an OmniRec `results.json` artifact was present; 'requested native metric' additionally requires at least one unique, finite nDCG or Precision entry at a requested cutoff. Native-result stages count checkpoint nodes only; one workspace-level result (P0/Mini/R03) counts at search level. The two 90-entry MovieLens nodes later failed protocol checks.**

still using a substitute computation or fabricated fallback output. Exception-free execution was necessary, but it was not sufficient to establish that the requested recommender experiment had been performed.

Native result production provides a stricter intermediate criterion. Twelve of the 19 searches (63.2%) produced at least one requested finite metric in an OmniRec `results.json` artifact, whereas seven produced none. Coverage was highly uneven. Some searches emitted only one or two requested entries; others advanced through several seeds or algorithms before a later component failed. No

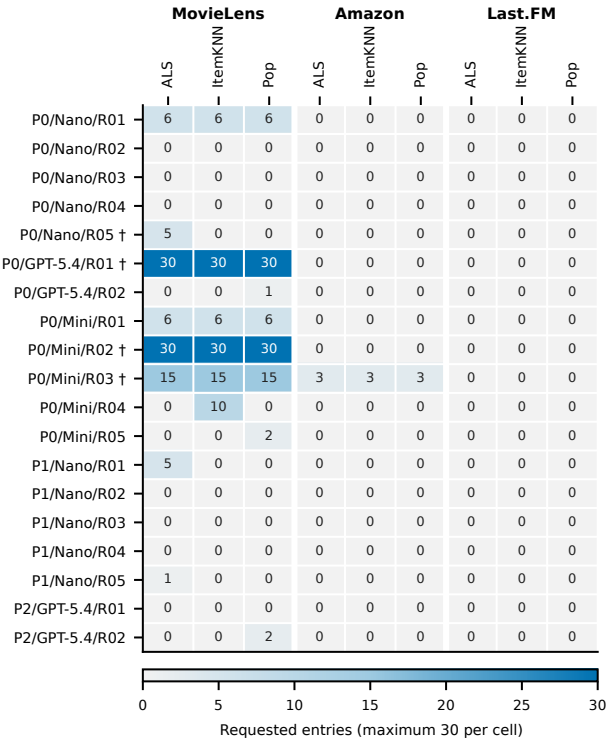


**Figure 3: Maximum coherent native requested coverage per dataset–algorithm cell for each production search (at most 30 entries per cell). Daggers mark documented semantic defects; unmarked coverage is not automatically valid. The two 90-entry MovieLens blocks failed protocol checks and the later datasets.**

search produced the complete matrix of three datasets, three algorithms, five seeds, three cutoffs, and two metrics within one coherent execution.

Two searches show why partial coverage must be preserved but not misclassified. In `V100_P0_MINI_I05_R02`—AutoRecLab v1.0.0, prompt P0, GPT-5.4 Mini, five search iterations, repetition 2—one node produced all 90 requested MovieLens entries, and in `V100_P0_GPT54_I05_R01` five seed-specific native files jointly provided a second 90-entry MovieLens execution; both then failed on Amazon. Real recommender components ran and native metrics were written, but preprocessing or split behavior deviated from the requested protocol, so neither is a semantically valid MovieLens reproduction and neither progressed toward the 270-entry result.

The outcome is therefore a completion funnel rather than a binary result: each stricter evidence stage removed cases that looked successful at the preceding stage, with the largest node-level drop before exception-free execution and further losses under semantic inspection. Reporting only 0/19 completion would hide this progress; reporting only executable nodes or result files would substantially overstate it (Figures 2 and 3).

### 3.3 Technical, Semantic and Orchestration Failure Modes

The artifacts reveal three interacting classes of failure. Technical failures prevented or interrupted execution; semantic failures produced outputs that did not represent the requested experiment; and orchestration failures affected how the search evaluated and selected its own nodes. These categories are non-exclusive: a single generated program could first alter the protocol to work around a framework error and then fail at a later dataset.

A recurring technical problem concerned split construction: several programs passed a zero-sized validation partition, which the installed stack rejected, while others avoided the exception by silently adding a non-requested validation partition, changing the required user-based 80/20 design. Loaders met column and schema assumptions that did not match the supplied MovieLens, Amazon, or Last.FM files; further failures involved downloads or built-in loaders instead of the supplied inputs, incorrect paths, OmniRec/LensKit API assumptions, and result objects outside the expected evaluation schema—amplified by the mismatch between P0's requirement for LensKit 0.14.4 and the installed environment. An excluded smoke test also failed on an approximately 3.64 GiB Amazon prediction allocation, showing that syntactically correct plans were not necessarily feasible.

Semantic errors were harder to detect because they produced plausible tables. The clearest preprocessing example, `MakeImplicit(3)`, retains ratings $\geq 3$ in the installed implementation (82,520 of the 100,000 MovieLens ratings), whereas the requested strict condition $> 3$ leaves 55,375 before core filtering; metrics computed after these operations refer to materially different interaction data even when the remaining code executes correctly.

Other cases changed the meaning of algorithms or metrics: `V100_P0_MINI_I05_R03` copied Recall rows and renamed them Precision, `V100_P0_NANO_I05_R05` presented a 135-row output under ALS, ItemKNN, and Pop labels while implementing a mock popularity recommender, and other nodes emitted zero-valued fallback tables after failures. Row counts and familiar column names were therefore insufficient evidence of algorithm identity or metric provenance.

Finally, tree-search finalization did not always retain the strongest node: some intermediate nodes contained native results or progressed further through the requested matrix than the node in the final response. This orchestration failure—useful work existed but was not selected or summarized reliably—justifies auditing intermediate nodes and persisting completed combinations incrementally. Table 3 links the observed failures to preventive checks.

### 3.4 Automated Reviewer Score Versus Objective Completion

AutoRecLab's reviewer score was not a reliable substitute for artifact completion. Across the 19 production searches, final requirement lists ranged from 11 to 22 items, with their number and wording varying by run and between the prototype and final stages. The best recorded reviewer scores ranged from 12.5% to 81.8%, yet no search received a true `is_satisfactory` marker and no search produced a complete valid result matrix. Because checklist sizes and contents varied, the same percentage did not necessarily represent the same experimental obligations across runs.

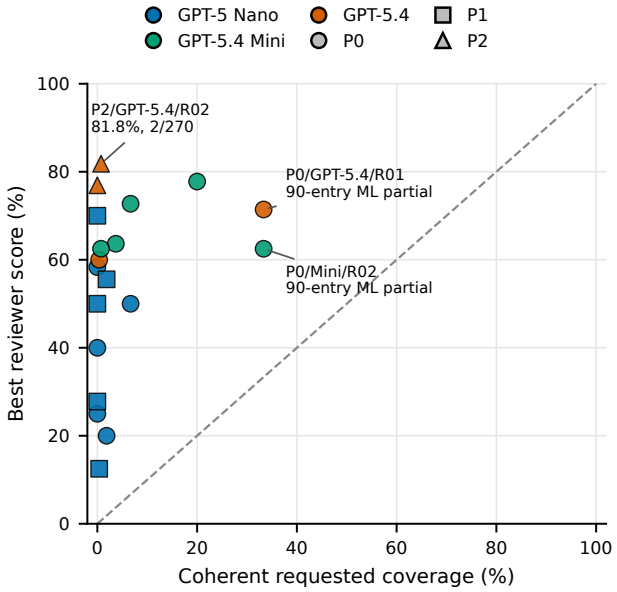


**Figure 4: Best automated reviewer score versus coherent requested native coverage (requested native entries from one coherent node or continued execution). Color encodes model, marker shape prompt. The association is descriptive because reviewer checklists varied; no point is a complete valid run.**

The association between best reviewer score and objective native coverage was weakly positive. We treat this relationship descriptively: there are only 19 searches, coverage is concentrated in a small number of partial executions, and the reviewer denominator is not fixed. More importantly, score magnitude could be badly misaligned for individual runs. The strongest example is `V100_P2_GPT54_I05_R02`, which achieved a best reviewer score of 81.8% while its selected coherent native result contained only two of the 270 requested entries, or approximately 0.7% coverage.

The mismatch arises because a language-model reviewer can reward code presence, stated intentions, filenames, or partial requirement satisfaction without proving that the requested combinations executed with the claimed semantics. Reviewer scoring remains useful as search feedback, but it should be paired with deterministic checks of the result matrix and semantic invariants before being read as scientific completion (Figure 4).

### 3.5 Cost, Runtime, and Valid Coverage

The 19 retained production searches made 2,832 recorded model API calls and cost USD 23.57. Summing the interval between the first and last entry in each cost log gives 910.52 minutes, or approximately 15.2 hours. This is a summed cost-log span, not an instrumented measure of CPU/GPU time: runs stopped at different failure stages, and the measure includes waiting and orchestration between recorded calls.

**Table 3: Observed failure mechanisms and corresponding validation checks.**

| Class | Mechanism | Observable consequence | Representative evidence | Detecting validation |
|---|---|---|---|---|
| Technical | Invalid split/validation configuration | Crash or changed 80/20 protocol | P0/Mini/R02 | Split-contract test |
| Technical | Dataset schema or loader mismatch | Input rejected or wrong fields used | P0/Mini/R04 | Schema preflight |
| Technical | OmniRec/LensKit API mismatch | Import, method, or result-schema failure | P0/Nano/R02 | Versioned API smoke test |
| Technical | Infeasible prediction allocation | Approximately 3.64 GiB allocation failure | Excluded smoke test | Resource-bound preflight |
| Semantic | Rating $\geq 3$ instead of $> 3$ | Different interaction population | P0/Mini/R02 | Post-filter count invariant |
| Semantic | Recall relabelled as Precision | Incorrect metric identity | P0/Mini/R03 | Metric-provenance check |
| Semantic | Mock recommender or zero fallback | Plausible shape without requested computation | P0/Nano/R05 | Class and value-provenance check |
| Semantic | Feedback type inferred from rating column | Implicit data run as rating prediction | Positive control; Last.FM | Explicit feedback declaration |
| Orchestration | Weaker final-node selection | Stronger partial evidence omitted | Multiple searches | Evidence-based node selection |
| Orchestration | Evaluation errors swallowed by runner | Empty results, no exception raised | Positive control | Result row-count assertion |
| Orchestration | Validation contract reinterpreted | Denominator changed 270 to 30; self-passed | P3/Mini/R03 | External non-editable validator |

Costs differed strongly by model. The ten Nano runs accounted for 1,537 calls, USD 2.14, and 600.77 logged minutes. The five GPT-5.4 Mini runs accounted for 655 calls, USD 3.78, and 178.01 minutes. The four GPT-5.4 runs accounted for 640 calls, USD 17.65, and 131.74 minutes, or 74.9% of total cost. At condition level, P0/Nano cost USD 1.10, P1/Nano USD 1.04, P0/Mini USD 3.78, P0/GPT-5.4 USD 7.73, and P2/GPT-5.4 USD 9.92. These totals describe the realized workload; they do not normalize for the point at which each search failed.

Higher expenditure did not guarantee greater valid coverage: the two P2/GPT-5.4 searches were the most expensive condition yet completed nothing, the 90-entry P0/GPT-5.4 partial remained methodologically invalid and failed on Amazon, and inexpensive Nano runs produced only small native partials. The evidence supports a cost-of-search rather than a cost-effectiveness interpretation: money bought additional generation and review attempts, while the bottlenecks were compatibility and semantic verification.

The design does not support a causal model comparison. Model, prompt, and software state were not fully crossed; P2 was adaptive; and run duration was partly determined by early failure. Per-run points and their achieved coverage are consequently more informative than means presented as if the conditions formed a balanced benchmark (Figure 5).

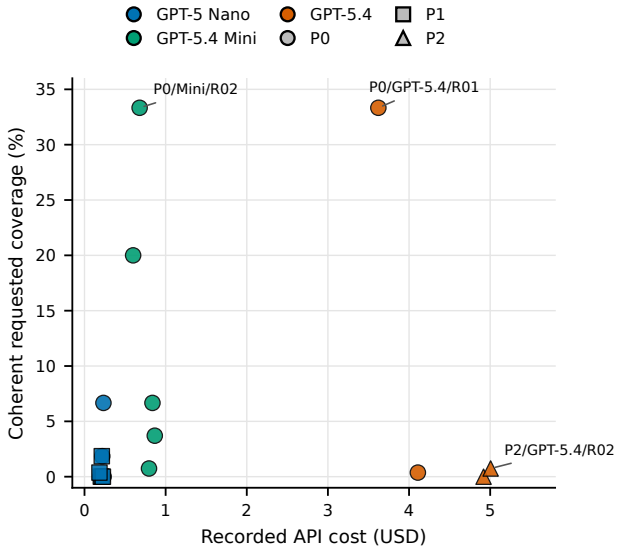


**Figure 5: Recorded API cost versus coherent requested native coverage (defined as in Figure 4). Color encodes model, marker shape prompt; no point is a complete valid run.**

## 3.6 Feasibility and Structural Blockers

The positive control produced all 270 unique, finite metric entries with zero semantic violations in 49.4 minutes of wall time and 737 MiB of peak memory, at no model-API cost. The requested experiment is therefore feasible in the final environment, and the absence of a valid autonomous completion is a reliability result rather than an infeasibility artifact. Constructing the control also isolated eight structural properties of the native path that a correct implementation must handle: an absent LensKit 0.14.4; a user-holdout splitter that cannot express an 80/20 split without a validation partition; an unbounded top-$N$ call that attempts an approximately 12.5 GiB ranking on Amazon Video Games; a checkpoint key derived only from split sizes, so all five seeds collide on one directory; an unseeded model-training path; an nDCG whose ideal DCG is not truncated to the number of relevant items, so a perfect ranking scores below one; a coordinator that catches and discards evaluation errors, returning empty results without raising an exception; and a runner that infers explicit feedback from the presence of a rating column, misrouting the already-implicit Last.FM data to rating prediction. Several of these reproduce, and explain, the failures observed in the autonomous runs (Table 3). The control also confirmed that the requested seed effect is real: across the five seeds the nDCG@10 of a fixed algorithm varied by 4–58% of its own value, largest where the data is sparsest.

## 3.7 Preregistered Intervention: P2 versus P3

Neither preregistered condition produced a valid completion: none of the five P2/Mini and none of the five P3/Mini searches yielded a coherent, semantically valid 270-entry matrix, and none received a true `is_satisfactory` marker. The two conditions used 820 and 803 model API calls and cost USD 5.58 and USD 6.34, respectively. The explicit contract did not repair completion. It did change behaviour, but not toward validity: P3 produced no native OmniRec `results.json` artifacts at all—against eight under P2—and instead emitted custom result tables with higher nominal key coverage but pervasive defects, including zero-valued entries, duplicate inflation, and algorithm-class provenance mismatches in which a Most Popular label was backed by an ItemKNN scorer class (Figure 6). The

**Table 4: Preregistered P2 versus P3 comparison on GPT-5.4 Mini, five runs each. No run in either condition was semantically valid.**

| Condition | Cost (USD) | Mean best reviewer | Native files | Complete valid |
|---|---|---|---|---|
| P2/Mini | 5.58 | 21.9% | 8 | 0/5 |
| P3/Mini | 6.34 | 12.1% | 0 | 0/5 |

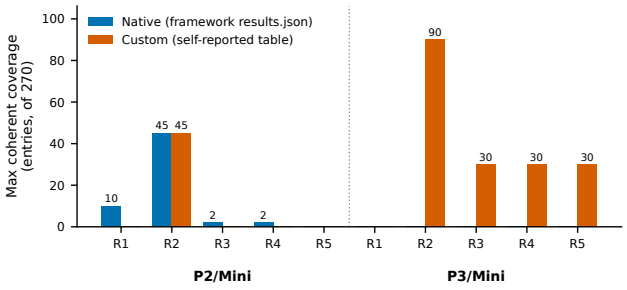


**Figure 6: Preregistered P2 versus P3 on GPT-5.4 Mini: maximum coherent requested coverage per run, split by source. P2 searches back their coverage with native OmniRec `results.json` artifacts; P3 searches produce none and report only self-generated tables with higher nominal but defective coverage. No run in either condition reached a valid 270-entry result.**

mean best reviewer score fell from 21.9% under P2 to 12.1% under P3 while completion remained zero in both (Table 4).

Most tellingly, the deterministic validation contract was reinterpreted rather than enforced. Among the P3 validation reports, one rewrote the expected denominator from 270 to 30 and reported `30/30` with zero failed checks, one reported `30/270` with zero failed checks, and one crashed while serialising a NumPy Boolean and left a corrupt report. Because the validator was generated code inside the agent's own workspace, the search could redefine or break it. An explicit contract is therefore necessary but not sufficient: deterministic validation must be external and non-editable to bind an autonomous search.

## 3.8 Discussion and Comparison with the Original Work

Our results do not reproduce the original 75% completion rate, but the gap has no single cause. The original estimate rests on four runs, so one run shifts it by 25 percentage points, whereas our 36 historical and 19 artifact-verified runs expose a wider range of failure modes. The original study's own 56% valid-subexperiment rate already showed that search-level completion can coexist with invalid components; our stricter result extends this observation by checking generated code and native outputs instead of accepting completion or reviewer statements alone.

Software evolution is a second major difference. The historical replication requested LensKit 0.14.4, while the later environment used OmniRec 1.0.0 with LensKit 2025.6.2, and generated programs frequently assumed a different API, dataset loader, split interface, or result schema. P2 removed some explicit incompatibilities but its two runs did not complete either. This does not show that newer software is intrinsically worse; it shows that an autonomous laboratory must verify the concrete environment before translating a natural-language protocol into executable code.

The central empirical distinction is between six outcomes generating code, exception-free execution, producing an output, producing requested native metrics, semantic validity, and completing the full matrix—with the reviewer score as a seventh, partially independent signal. A system can advance at one level and fail at the next: a mock recommender can run and emit a correctly shaped table, while a genuine OmniRec execution can cover one dataset and fail on the next. The conclusion would change substantially depending on which level is called "success"; artifact-verified semantic completion is the appropriate endpoint when the output is meant to support a scientific claim.

Partial progress nevertheless matters. The two 90-entry MovieLens executions show that AutoRecLab could assemble and run substantial parts of the requested pipeline, and the generated nodes reveal concrete interface problems and candidate corrections for a subsequent iteration. The limitation is not an absolute inability to generate recommender experiments but unreliable end-to-end control over preprocessing, interfaces, metric semantics, and completion across a complex matrix—which keeps the retained intermediate artifacts informative even though they cannot answer the original random-seed question.

The experiment isolates neither prompt nor model quality. P1 ran only with Nano and P2 only with GPT-5.4; the P0 conditions had unequal repetitions and costs; P2 was formulated after P0 failures and incorporates information unavailable to earlier conditions; and software state and team environment vary across phases. The single historical simplified-prompt success therefore cannot establish that less detail is better, nor can the larger partial coverage of some Mini or GPT-5.4 runs establish model superiority.

Taken together, the findings shift the evaluation of autonomous experimentation from surface completion to evidence integrity. AutoRecLab's planner, coder, and reviewer generate considerable experimental work, but their shared language-model representation of the task provides no independent semantic check. Deterministic contracts for data, splits, algorithms, metrics, and expected outputs are needed alongside generative search if the artifacts are to support reproducible conclusions.

Two additions sharpen this conclusion. First, the human-authored control shows that the specification is achievable in the exact environment at negligible cost, so the autonomous failures reflect unreliable end-to-end control rather than an impossible or internally inconsistent task. Second, the preregistered intervention shows that telling the system what is incompatible and requiring it to validate its own output is not enough: the compatibility contract did not raise completion, and the deterministic validator was reinterpreted by the generated code—in one run its denominator was silently reduced from 270 to 30. Surface signals of success—executable code, populated tables, a passing self-report, an approving reviewer—were each produced without a valid experiment. Reliable autonomous experimentation therefore requires deterministic checks that are

external to the agent and cannot be edited by the search, applied to data, splits, algorithms, metrics, and the expected result matrix.

### 3.9 Suggested Improvements

The first improvement is a mandatory pre-execution compatibility stage that records available package versions and algorithm classes, loads a small sample from every supplied dataset, validates column mappings, tests the requested split and evaluator APIs, and estimates prediction and model artifact sizes. These checks directly address the observed version mismatches, schema and path errors, rejected validation settings, and the memory failure.

Second, the natural-language task should be compiled into a machine-readable experiment contract. In this experiment the contract would contain the three input checksums, strict rating threshold, iterative five-core rule, user-based 80/20 split, accepted algorithm classes, five seeds, metrics, cutoffs, and the 270 expected unique result keys. A validator should compare emitted preprocessing and split counts with these invariants before accepting any metric, and metric provenance should link each value to the executed algorithm and evaluator—detecting the threshold error, mock substitution, Recall-to-Precision relabeling, duplicate keys, non-finite values, and zero fallbacks. The validator should execute outside the agent's writable workspace and use a fixed, non-editable specification, because the P3 runs showed that an in-band validator could be redefined, weakened, or broken by generated code.

Third, reviewer requirements should remain fixed for the mandatory parts of the task and be evaluated against deterministic evidence: matrix coverage, execution status, semantic-validation results, and artifact paths rather than generated code and narrative summaries alone. Final-node selection should prioritize the strongest validated artifact and preserve better intermediate nodes, addressing both the varying checklist denominators and the observed reviewer–coverage divergence. For high-cost runs, an optional human checkpoint between planning and code generation could reject an incompatible design early, at the cost of full autonomy.

Finally, every run should record standardized provenance—prompt hash, repository commit, resolved configuration, dependency lockfile, dataset checksums, node lineage, model identifier, costs, timestamps, code, logs, and native results—and write results incrementally after each coherent combination. This would make software-state comparisons explicit, prevent completed partials from being lost after later failures, and allow automated summaries to be reconstructed from primary artifacts rather than trusted as the only record.

### 3.10 Overall Finding

AutoRecLab frequently generated executable code and, in 12 of 19 retained production searches, produced at least one requested native metric. It also produced two substantial 90-entry MovieLens partial executions. However, no retained search translated the complete specification into a coherent, semantically valid 270-entry experiment. Apparent success at the levels of code execution, output presence, or reviewer score therefore did not imply a valid reproduction.

Within the evaluated task and software environments, the answer to whether AI can run its own experiments is consequently conditional: it can autonomously produce and execute meaningful parts of an experiment, but it did not reliably verify or complete the experiment without artifact-level checks. The main contribution of this replication is the demonstration that autonomous scientific systems must be evaluated not only by whether they run, but by whether their retained artifacts prove that the intended experiment was actually performed.

## 4 LIMITATIONS

### 4.1 Missing and Incomplete Artifacts

The principal limitation is unequal artifact availability across phases. The complete workspaces of the 36 historical replication runs are no longer available. Only aggregate outcomes and eight partially recovered records remain, and the single historically reported simplified-prompt success cannot be checked against the semantic criteria used for the retained follow-up. Consequently, the historical phase supports a comparison of reported outcomes, not an artifact-level reanalysis equivalent to that of the 19 fully retained production searches. The original study's reported outcomes are likewise taken from its publication rather than independently re-audited.

The four `develop`-branch runs are treated as a separate evidence tier because their complete workspaces were not part of the primary artifact corpus. They are therefore used only where their preserved records support a claim and are excluded from the 19-run denominator. Within the retained corpus, the audit opened and hashed every file, but serialized pickle and model objects were not deserialized because doing so can execute untrusted code. Large prediction files were inspected structurally and with streaming procedures. These choices preserve safety and permit coverage analysis, but they can miss state available only inside serialized objects. Finally, file hashes establish identity and reveal duplicates; they do not by themselves establish correct provenance or semantic validity.

### 4.2 Experimental Design and Environment

The follow-up was not a randomized, fully crossed prompt–model experiment. P1 was evaluated only with Nano, P2 only with GPT-5.4, and P0 conditions had unequal repetitions and costs. P2 was designed after observing P0 failures and therefore incorporates information unavailable to earlier conditions. Prompt, model, software state, and in some cases machine or branch consequently vary together, so their individual effects cannot be separated. The small number of runs per condition and the absence of a valid completion further preclude claims that one model or prompt is superior.

The environments also differed from the original work and from each other. The historical specification referred to LensKit 0.14.4, whereas the 19 fully retained runs used OmniRec 1.0.0 with LensKit 2025.6.2; the separately reported four-run set used the `develop` implementation. Hardware, existing caches, downloaded archives, and dataset representations were not held constant across every phase. Some generated programs used built-in loaders or downloaded copies instead of the supplied inputs. These differences are themselves relevant reliability problems, but they prevent a controlled estimate of the effect of software evolution.

We did not systematically optimize recommender hyperparameters because the primary question concerned faithful construction of the specified experiment, and most searches failed before a complete comparable benchmark existed. The requested five seeds were attempted, but no coherent, semantically valid three-dataset matrix completed; the study therefore cannot evaluate seed sensitivity or recommender-performance differences. A human-authored positive control was executed under the final environment and completed the full 270-entry matrix, so infeasibility is excluded as an explanation for the autonomous failures; the control is nonetheless a single pipeline and does not evaluate recommendation quality. The preregistered P2–P3 comparison is balanced and fixed in advance, but it uses one model (GPT-5.4 Mini) with five repetitions per condition, and some repetitions ran concurrently on shared hardware, so its runtime and peak-memory figures are descriptive only; its cost, coverage, semantic-validity, and completion results are unaffected by that concurrency. Cost-log spans include orchestration and waiting time, are not direct compute-time measurements, and no controlled energy or $CO_2$ measurement was collected.

### 4.3 Scope and Validity of the Analysis

The 188 generated code nodes are nested attempts within 19 searches, not 188 independent observations. Node-level execution percentages describe the search process and must not be interpreted as independent estimates of system success. The 19-run sample is also too small and unbalanced for strong inferential comparisons. Wilson intervals in Figure 1 communicate uncertainty in observed run-level proportions, but the phases use different software states, evidence quality, and success assessments; the intervals do not make the phase comparison controlled.

Native metric coverage is an intentionally narrow intermediate measure. It counts finite requested entries in a coherent retained result artifact but does not prove correct preprocessing, split construction, algorithm identity, or metric semantics. Semantic assessment required manual comparison of generated code, logs, and outputs, and some classifications therefore involve human judgement. The criteria were derived from the task specification and applied consistently, but no independent second audit or formal inter-rater agreement was performed. Mocks, relabelled metrics, and threshold deviations demonstrate why this judgement was necessary, while also limiting the precision of any failure-frequency claims. The qualitative failure taxonomy should therefore be read as a set of observed mechanisms rather than an exhaustive or mutually exclusive prevalence estimate.

Finally, the evaluation covers one autonomous system, one complex recommender-systems task, the available model variants, and the software environments retained in this project. It does not establish reliability for other scientific domains, tasks, agent architectures, or current and future model versions. Because no valid full experiment completed, the work evaluates the integrity of autonomous experiment generation rather than the substantive recommender question posed by the original protocol. Its strongest conclusion is correspondingly bounded: within this case study, execution, output presence, and automated reviewer approval were insufficient evidence of semantically valid completion.

## 5 CODE AND DATA AVAILABILITY

Code, prompts, resolved configurations, audit tables, analysis scripts, the positive-control pipeline, and the external validator are publicly available in [6], at commit `ea45f96` as of 24 July 2026; an immutable reference is provided with a tagged release available at https://github.com/dylanfandio/AutoRecLab/releases/tag/study-v1.0.0.

The original datasets—MovieLens 100K [3], Amazon Video Games [5], and Last.FM [2]—are not redistributed. Their source locations and SHA-256 checksums are provided. The complete run workspaces are not publicly hosted because of their size. A complete per-file SHA-256 manifest and a curated evidence subset covering the central case studies discussed in this paper are included regardless. Aggregate outcomes for the 36 historical replication runs are reported as provided by the retained records. The corresponding raw workspaces are, with the exception of eight partially recovered records, no longer available and are not part of this release.

## REFERENCES


[1] Joeran Beel, Bela Gipp, Tobias Vente, Moritz Baumgart, and Philipp Meister. 2025. From AutoRecSys to AutoRecLab: A Call to Build, Evaluate, and Govern Autonomous Recommender-Systems Research Labs. arXiv:2510.18104 [cs.IR] https://arxiv.org/abs/2510.18104

[2] GroupLens Research. [n. d.]. HetRec 2011 Last.FM 2K Dataset. https://grouplens.org/datasets/hetrec-2011/. Accessed: 2026-07-22.

[3] GroupLens Research. [n. d.]. MovieLens 100K Dataset. https://grouplens.org/datasets/movielens/100k/. Accessed: 2026-07-22.

[4] Information Systems Group, University of Siegen. 2026. AutoRecLab Source Code. https://github.com/ISG-Siegen/AutoRecLab. Accessed: 2026-07-22.

[5] Julian McAuley and collaborators. [n. d.]. Amazon Review Data: Amazon Reviews 2018. https://cseweb.ucsd.edu/~jmcauley/datasets/amazon_v2/. Accessed: 2026-07-22.

[6] Team 2. 2026. AutoRecLab Replication Repository. https://github.com/dylanfandio/AutoRecLab. Accessed: 2026-07-22.